

ICPC Code Template

喵喵喵幼儿园

8 CATEGORIES

25 TEMPLATES

1163 SOURCE LINES

Quick Index

1 Data Structures	2
1.1 01 树状数组	2
1.2 O(1) 查询树状数组	2
1.3 Top Tree	3
2 Geometry	3
2.1 Geometry basic	3
3 Graph / Flow	4
3.1 最大流	5
3.2 最大流	5
3.3 最小费用最大流	6
4 Math	6
4.1 Binary GCD	7
4.2 det	7
5 Math / Number Theory	7
5.1 Euclid	7
5.2 exgcd	8
5.3 min25	8
6 Misc	9
6.1 Default(Yzj)	9
6.2 快速模乘	9
7 Poly	9
7.1 多项式求逆	9
7.2 多项式乘法	11
7.3 计数常用代码	12
7.4 DIF-DIT NTT & All op	13
8 String	16
8.1 广义 SAM	16
8.2 manacher	17
8.3 PAM	17
8.4 SA	17
8.5 后缀数组	18
8.6 SAM	18
8.7 后缀树	19

1 Data Structures

1.1 01 树状数组

FILE data-structures/fwt_bitset.cpp

LINES 41

USE 下标从 1 开始，激活位置并查询区间内已激活位置数

COST Add/query $O(\log(N/64))$, in-block query $O(1)$

AUTHOR ppip

```

1 struct fwt {
2     static constexpr int B{(N>>6)+5};
3     int t[B],n;
4     unsigned long long b[B];
5     void clear() {
6         n=(n>>6)+1;
7         fill_n(t,n+1,0);
8         fill_n(b,n+1,0);
9     }
10    int px(int x) {
11        int y{x&63},z{x>>6};
12        return __builtin_popcountll(b[z]&(~0ull>>(63-y)));
13    }
14    int qry(int l,int r) {
15        --l;
16        int ans{px(r)-px(l)};
17        l>>=6;r>>=6;
18        while (r>l) ans+=t[r],r&=r-1;
19        while (l>r) ans-=t[l],l&=l-1;
20        return ans;
21    }
22    void add(int x) {
23        unsigned long long w{1ull<<(x&63)};
24        if (b[x>>6]&w) return;
25        b[x>>6]|=w;
26        x=(x>>6)+1;
27        while (x<=n) {
28            ++t[x];
29            x+=x&-x;
30        }
31    }
32    int qry(int x) {
33        int ans{px(x)};
34        x>>=6;
35        while (x) {
36            ans+=t[x];
37            x&=x-1;
38        }
39        return ans;
40    }
41 } t;

```

1.2 $O(1)$ 查询树状数组

FILE data-structures/fwt_suffix.cpp

LINES 16

USE 下标从 1 开始，单点加、查询后缀和；适合查询远多于修改

COST Query $O(1)$, add $O(128+N/4096)$

AUTHOR ppip

```

1 struct FWT {
2     int s1[N+5],s2[(N>>6)+5],s3[(N>>12)+5];
3     void init() {
4         memset(s1,0,sizeof(s1));
5         memset(s2,0,sizeof(s2));
6         memset(s3,0,sizeof(s3));
7     }
8     void add(int x,int y) {
9         for (int i{(x>>6)<<6};i<=x;++i) s1[i]+=y;
10        for (int i{(x>>12)<<6};i<(x>>6);++i) s2[i]+=y;
11        for (int i{0};i<(x>>12);++i) s3[i]+=y;
12    }
13    int qry(int x) {
14        return s1[x]+s2[x>>6]+s3[x>>12];
15    }
16 } T;

```

1.3 Top Tree

FILE data-structures/toptree.cpp

LINES 60

USE Magic tree that can fulfill every wish.

COST Tree height $O(\log n)$

AUTHOR ppip

```

1 vector<pair<int,int>> e[N+5];
2 int sz[N+5],fa[N+5],son[N+5];
3 void TOPTREE_INIT(int u) {
4     sz[u]=1;
5     for (auto [v, _]:e[u])
6         if (v!=fa[u]) {
7             TOPTREE_INIT(v);
8             sz[u]+=sz[v];
9             if (sz[v]>sz[son[u]]) son[u]=v;
10        }
11    if (son[u]) {
12        int si{exchange(sz[son[u]],0)};
13        sort(e[u].begin(),e[u].end(),[](auto x,auto y){return sz[x.first]<sz[y.first]});
14        sz[son[u]]=si;
15    }
16 }
17 struct cluster {
18     //....
19 } t[N*2+5];
20 int cnode;
21 using P=pair<int,int>;
22 using iter=vector<P>::iterator;
23 int merge(iter L,iter R,char tp) {
24     if (L+1==R) {
25         int z{L->first};
26         if (z<=n) t[z].u=fa[z],t[z].v=z;
27         return z;
28     }
29     iter p{L+1};
30     int sum{accumulate(p,R,-L->second,[](int x,P y){return x+y.second;})};
31     while (p+1!=R&&sum>p->second) {
32         sum-=p->second*2;
33         ++p;
34     }
35     int u{++cnode};
36     t[u].t=tp;
37     t[u].l=merge(L,p,tp);
38     t[u].r=merge(p,R,tp);
39     t[t[u].l].f=t[t[u].r].f=u;
40     t[u].u=t[t[u].l].u;
41     t[u].v=(tp=='C'?t[t[u].r].v:t[t[u].l].v);
42     return u;
43 }
44 int build(int u) {
45     vector<P> v1;v1.emplace_back(u,1);
46     while (son[u]) {
47         vector<P> v2;v2.emplace_back(son[u],1);
48         for (auto [v, _]:e[u])
49             if (v!=fa[u]&&v!=son[u]) v2.emplace_back(build(v),sz[v]);
50         v1.emplace_back(merge(v2.begin(),v2.end(),'R'),sz[u]-sz[son[u]]);
51         u=son[u];
52     }
53     return merge(v1.begin(),v1.end(),'C');
54 }
55 int main() {
56     cnode=n;
57     TOPTREE_INIT(1);
58     int rt{build(1)};
59     return 0;
60 }

```

2 Geometry

2.1 Geometry basic

FILE Geometry/basic.cpp

LINES 68

USE 凸包, 检查一个点是否在凸包内, 闵可夫斯基和

AUTHOR yzj

```

1 // @complexity
2 namespace Geometry {
3     const db eps = 1e-5;
4     struct Vector {
5         ll x,y;
6         Vector operator+(const Vector &A) { return Vector{x+A.x,y+A.y}; }
7         Vector operator-(const Vector &A) { return Vector{x-A.x,y-A.y}; }
8         Vector operator*(const Vector &A) { return Vector{x*A.x,y*A.y}; }
9         bool operator==(const Vector &A) { return x==A.x && y==A.y; }
10        inline ll square() { return x*x+y*y; }
11        inline db dis() { return sqrt((db)(x*x+y*y)); }
12        inline int section() {
13            if(x>0 && y>=0) return 1;
14            else if(x<=0 && y>0) return 2;
15            else if(x<0 && y<=0) return 3;
16            else return 4;
17        }
18        inline void in() { x = read(); y = read(); }
19    };
20    const Vector zero = {0,0};
21    vc<Vector> as;
22
23    inline ll crs(Vector A,Vector B) { return A.x*B.y-A.y*B.x; }
24    inline bool right(Vector A,Vector B) { return crs(A,B)<0; }
25    inline bool left (Vector A,Vector B) { return crs(A,B)>0; }
26    inline bool cmp1(Vector A,Vector B) { return A.y==B.y?A.x<B.x:A.y<B.y; }
27    inline bool cmpangle(Vector A,Vector B) { return A.section()==B.section()?
left(A,B):A.section()<B.section(); }
28    inline bool inseg(Vector A,Vector B,Vector C) {
29        return fabs((C-A).dis()+(C-B).dis()-(B-A).dis())<eps;
30    }
31
32    inline vc<Vector> ConvexHull(vc<Vector> a) {
33        sort(a.begin(),a.end(),cmp1);
34        vc<Vector> b, c, o; int l=0;
35        for(auto u:a) { while(b.size()>1 && right(b[l-1]-b[l-2],u-b[l-1])) b.pop_back(), --l; b.pb(u); +
+1; }
36        l=0; reverse(a.begin(),a.end());
37        for(auto u:a) { while(c.size()>1 && right(c[l-1]-c[l-2],u-c[l-1])) c.pop_back(), --l; c.pb(u); ++l;}
38        o=b; for(int i=1;i<l-1;i++) o.pb(c[i]);
39        return o;
40    }
41
42    inline vc<Vector> Minkowski(vc<Vector> A,vc<Vector> B) {
43        if(!A.size()) return A; if(!B.size()) return B;
44        vc<Vector> C; Vector s = A[0]+B[0];
45        for(int i=0;i<A.size();i++) C.pb(A[(i+1)%A.size()]-A[i]);
46        for(int i=0;i<B.size();i++) C.pb(B[(i+1)%B.size()]-B[i]);
47        sort(C.begin(),C.end(),cmpangle);
48        vc<Vector> res; Vector sum = s; res.pb(s);
49        for(auto u:C) sum = sum+u, res.pb(sum);
50        return res.pop_back(), res;
51    }
52
53    inline bool in(Vector x) { // ask whether x is in ConvexHull as
54        if(as.size()==0) return 0;
55        if(as.size()==1) return as[0]==x;
56        if(as.size()==2) return !crs(as[l-1]-as[0],x-as[0]) && inseg(as[l-1],as[0],x);
57        int l = as.size();
58        if(left(as[l-1]-as[0],x-as[0])) return 0;
59        int le = 1, ri = l-2, mid, cnt=0;
60        while(le <= ri) {
61            ++cnt;
62            mid = le+ri>>1;
63            if(!right(as[mid]-as[0],x-as[0])) le = mid+1;
64            else ri = mid-1;
65        }
66        return !right(as[ri+1]-as[ri],x-as[ri]);
67    }
68 }

```

3 Graph / Flow

3.1 最大流

FILE graph/flow/maxflow(yzj).cpp

LINES 27

USE 网络最大流

COST $O(nm^2)$, 二分图中 $O(m \sqrt{n})$

AUTHOR yzj

```

1 namespace flow {
2     int dep[N], S, T;
3     ll maxflow;
4     int hd[N], cur[N], to[N << 1], ne[N << 1], e[N << 1], tc = 1;
5     inline void add(int x, int y, int z) {
6         to[++tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z;
7         to[++tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0;
8     }
9     inline int bfs() {
10         rep(i, 0, T) dep[i] = -1; dep[S] = 0; queue<int> Q; Q.push(S);
11         while(!Q.empty()) {
12             int x = Q.front(); Q.pop(); cur[x] = hd[x];
13             if(x == T) return 1;
14             for(int i = hd[x], y; i; i = ne[i])
15                 if(e[i] && dep[y = to[i]] == -1) dep[y] = dep[x] + 1, Q.push(y);
16         }
17         return 0;
18     }
19     inline int dfs(int x, int fl) {
20         if(x == T) return maxflow += fl, fl;
21         for(int &i = cur[x], y, o; i; i = ne[i])
22             if(e[i] && dep[y = to[i]] == dep[x] + 1 && (o = dfs(y, min(fl, e[i]))))
23                 return e[i] -= o, e[i ^ 1] += o, o;
24         return 0;
25     }
26     inline void dinic() { while(bfs()) { while(dfs(S, inf)); } }
27 }

```

3.2 最大流

FILE graph/flow/maxflow.cpp

LINES 60

USE Dinic, 要求汇点编号为最大值

COST 相信的心就是魔法

AUTHOR ppip

```

1 struct edge {
2     int u,v,w,nxt;
3     edge(int __,int __,int __,int __):
4         u{__},v{__},w{__},nxt{__} {}
5     edge() {}
6 };
7 vector<edge> e(2);
8 int hd[N+5];
9 void _add(int u,int v,int w) {
10     e.emplace_back(u,v,w,hd[u]);
11     hd[u]=e.size()-1;
12 }
13 void add(int u,int v,int w) {
14     _add(u,v,w);
15     _add(v,u,0);
16 }
17 int q[N+5];
18 int dep[N+5],cur[N+5];
19 bool bfs(int s,int t) {
20     dep[s]=1;
21     int l{0},r{0};
22     q[r++]=s;
23     while (r-l) {
24         int u{q[l++]};
25         for (int x{hd[u]};x;x=e[x].nxt)
26             if (!dep[e[x].v]&&e[x].w) {
27                 dep[e[x].v]=dep[u]+1;
28                 q[r++]=e[x].v;
29             }
30     }
31     return dep[t];
32 }
33 long long dfs(int s,int t,long long flow) {
34     if (s==t||!flow) return flow;
35     long long ans{0};
36     for (int &x{cur[s]},d;x;x=e[x].nxt) {
37         if (dep[e[x].v]==dep[s]+1&&e[x].w&&(d=dfs(e[x].v,t,min(flow,(long long)e[x].w)))) {

```

```

38         flow-=d;
39         ans+=d;
40         e[x].w-=d;
41         e[x^1].w+=d;
42         if (!flow) return ans;
43     }
44 }
45 return ans;
46 }
47 long long calc(int s,int t) {
48     fill_n(dep+1,t,0);
49     long long flow{0};
50     while (bfs(s,t)) {
51         copy(hd+1,hd+t+1,cur+1);
52         flow+=dfs(s,t,LLONG_MAX);
53         fill_n(dep+1,t,0);
54     }
55     return flow;
56 }
57 void clear(int t) {
58     e.resize(2);
59     fill_n(hd+1,t,0);
60 }

```

3.3 最小费用最大流

FILE graph/flow/mcmf.cpp

LINES 38

COST $O(nmF)$

AUTHOR yzj

```

1 // @brief
2 namespace MCMF {
3     const int MAXN = 1e6 + 5, inf = 1e18;
4     int s, t, mxfl, mico;
5     int inq[MAXN], d[MAXN], pr[MAXN];
6     int hd[MAXN], cur[MAXN], to[MAXN << 1], ne[MAXN << 1], e[MAXN << 1], c[MAXN << 1], tc = 1;
7
8     inline void add (int x, int y, int z, int w) {
9         // cerr << x << " " << y << " " << z << " " << w << endl;
10        to[++ tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z; c[tc] = w;
11        to[++ tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0; c[tc] = - w;
12    }
13
14    inline int spfa () {
15        rep (i, 0, t) inq[i] = pr[i] = 0, d[i] = inf;
16        queue <int> Q; Q.push (s); d[s] = 0; inq[s] = 1;
17        while (!Q.empty ()) {
18            int x = Q.front (); Q.pop (); inq[x] = 0; cur[x] = hd[x];
19            for (int i = hd[x], y; i; i = ne[i])
20                if (e[i] && d[y = to[i]] > d[x] + c[i]) {
21                    d[y] = d[x] + c[i]; pr[y] = i;
22                    if (!inq[y]) Q.push (y), inq[y] = 1;
23                }
24        }
25        return d[t] != inf;
26    }
27
28    inline pii mcmf() {
29        pii res = mkp(0, 0);
30        while(spfa ()) {
31            int c = inf;
32            for(int x = t; x != s; x = to[pr[x] ^ 1]) c = min(c, e[pr[x]]);
33            for(int x = t; x != s; x = to[pr[x] ^ 1]) e[pr[x]] -= c, e[pr[x] ^ 1] += c;
34            res.first += c; res.second += d[t] * c;
35        }
36        return res;
37    }
38 }

```

4 Math

4.1 Binary GCD

FILE math/binary_gcd.cpp

LINES 6

USE 超快 gcd 算法

COST $O(\log n)$, 可以当作 $O(1)$

AUTHOR ppip

```

1 #define ctz __builtin_ctz
2 inline int gcd(int a,int b) {
3     int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
4     while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
5     return b<<z;
6 }

```

4.2 det

FILE math/det.cpp

LINES 17

USE 行列式求值

COST $O(n^3)$

AUTHOR yzj

```

1 inline mint det (vc <vc <mint> > a, int n) {
2     int rev = 0;
3     rep (i, 1, n) {
4         rep (j, i + 1, n) {
5             while (a[i][i].x) {
6                 int d = a[j][i].x / a[i][i].x;
7                 rep (k, i, n) a[j][k] -= a[i][k] * d;
8                 swap (a[i], a[j]); rev ^= 1;
9             }
10            swap (a[i], a[j]); rev ^= 1;
11        }
12        if (!a[i][i].x) return 0;
13    }
14    mint res = 1;
15    rep (i, 1, n) res *= a[i][i];
16    return rev ? MOD - res : res;
17 }

```

5 Math / Number Theory

5.1 Euclid

FILE math/number_theory/euclid.cpp

LINES 44

COST $O(\log \max(a, c))$

AUTHOR yzj

```

1 // @brief
2 namespace Euclid {
3     const int mod = 998244353, inv2 = 499122177, inv6 = 166374059;
4     struct Data { int f, g, h; };
5
6     inline Data calc(int n, int a, int b, int c) {
7         if(a >= c || b >= c) {
8             int A = a / c, B = b / c;
9             Data t = calc(n, a % c, b % c, c);
10            int N = n % mod, s0 = (N + 1) % mod;
11            int s1 = N * s0 % mod * inv2 % mod;
12            int s2 = N * s0 % mod * (2 * N + 1) % mod * inv6 % mod;
13            A %= mod; B %= mod;
14
15            int f = (t.f + A * s1 + B * s0) % mod;
16            int g = (t.g + A * A % mod * s2 + B * B % mod * s0
17                + 2 * A % mod * t.h + 2 * B % mod * t.f
18                + 2 * A % mod * B % mod * s1) % mod;
19            int h = (t.h + A * s2 + B * s1) % mod;
20            return {f, g, h};
21        }
22
23        int m = (a * n + b) / c;
24        if(!m) return {0, 0, 0};
25
26        Data t = calc(m - 1, c, c - b - 1, a);
27        int N = n % mod, M = m % mod;
28        int s1 = N * ((N + 1) % mod) % mod * inv2 % mod;
29
30        int f = (N * M % mod - t.f) % mod;
31        int g = (N * M % mod * M % mod - 2 * t.h - t.f) % mod;
32        int h = (M * s1 % mod - (t.g + t.f) % mod * inv2) % mod;

```

```

33         if(f < 0) f += mod;
34         if(g < 0) g += mod;
35         if(h < 0) h += mod;
36         return {f, g, h};
37     }
38 }
39
40 auto [f, g, h] = Euclid::calc(n, a, b, c);
41
42 f // sum floor((ai+b)/c)
43 g // sum floor((ai+b)/c)^2
44 h // sum i*floor((ai+b)/c)

```

5.2 exgcd

FILE math/number_theory/exgcd.cpp

LINES 12

COST $O(\log n)$

AUTHOR yzj

```

1 // @brief
2 inline int exgcd(int a, int b, int &x, int &y) {
3     if(!b) return x = 1, y = 0, a;
4     int d = exgcd(b, a % b, y, x);
5     y -= a / b * x;
6     return d;
7 }
8 inline int inv(int a, int mod) {
9     int x, y;
10    exgcd(a, mod, x, y);
11    return (x % mod + mod) % mod;
12 }

```

5.3 min25

FILE math/number_theory/min25.cpp

LINES 49

COST $O(n^{3/4} / \log n)$

AUTHOR yzj

```

1 // @brief
2 namespace min25 {
3     int pri[N], cov[N], sq, m;
4     int id1[N], id2[N], val[N], idx;
5     mint f[N], g[N], h[N];
6     map<pii, mint> dps;
7
8     inline int id (int x) { return x <= sq ? id1[x] : id2[n / x]; }
9
10    inline void init (int n) {
11        sq = sqrt (n); m = idx = 0;
12
13        rep (i, 2, sq) {
14            if (!cov[i]) pri[++ m] = i, h[m] = h[m - 1] + i;
15            for (int j = 1; j <= m && i * pri[j] <= sq; j++) {
16                cov[i * pri[j]] = 1;
17                if (i % pri[j] == 0) break;
18            }
19        }
20
21        for (int l = 1, r, v; l <= n; l = r + 1) {
22            val[++ idx] = v = n / l; r = n / v;
23            (v <= sq ? id1[v] : id2[n / v]) = idx;
24            f[idx] = (mint) v * (v + 1) * inv2 - 1; g[idx] = v - 1;
25        }
26
27        rep (j, 1, m) for (int i = 1; i <= idx && pri[j] * pri[j] <= val[i]; i++) {
28            int k = id (val[i] / pri[j]);
29            f[i] -= pri[j] * (f[k] - h[j - 1]);
30            g[i] -= g[k] - j + 1;
31        }
32    }
33
34    int TIME = 0;
35    inline mint S (int n, int i) {
36        if (n <= 1 || pri[i] >= n) return 0;
37
38        int u = id (n);
39        mint res = f[u] - g[u] - h[i] + i;

```

```

40     if (i == 0) res += 2;
41
42     for (int j = i + 1; j <= m && pri[j] * pri[j] <= n; j++) {
43         for (int c = 1, pw = pri[j]; pw <= n; pw *= pri[j], c++)
44             res += (pri[j] ^ c) * (S(n / pw, j) + (c > 1));
45     }
46
47     return res;
48 }
49 }

```

6 Misc

6.1 Default(Yzj)

FILE misc/default(yzj).cpp

LINES 24

AUTHOR 喵喵喵幼儿园

```

1  #include<bits/stdc++.h>
2
3  #define int long long
4
5  #define vc vector
6  #define pb emplace_back
7  #define pii pair<int, int>
8  #define mkp make_pair
9  #define rep(i, a, b) for(int i = (a); i <= (b); ++i)
10 #define lep(i, a, b) for(int i = (a); i >= (b); --i)
11
12 using namespace std;
13
14 mt19937 gen(chrono::system_clock::now().time_since_epoch().count());
15
16 inline int read() {
17     int x = 0, w = 0; char ch = getchar(); while(!isdigit(ch)) w |= (ch == '-'), ch = getchar();
18     while(isdigit(ch)) x = x * 10 + (ch ^ 48), ch = getchar(); return w ? -x : x;
19 }
20
21 signed main() {
22
23     return 0;
24 }

```

6.2 快速模乘

FILE misc/fastmod.cpp

LINES 16

USE 要求 $1 \leq P < 2^{31}$ 且 $a < P$; 同一 z 多次使用时先 trans, 再用 mul_tCOST $O(1)$

AUTHOR ppip

```

1  using u32=unsigned;
2  using u64=unsigned long long;
3  using u128=unsigned __int128;
4  const u32 P=998244353;
5  inline u64 trans(u64 x) {
6      constexpr u64 A=-(u64)P/P+1;
7      constexpr u64 q=((u128(-(u64)P%P)<<64)+P-1)/P;
8      return x*A+u64((u128)x*q>>64)+1;
9  }
10 // t 必须是 trans(z) 的返回值; 复用同一 z 时只需计算一次 t
11 inline u32 mul_t(u32 a,u64 t) {
12     return a*t*(u128)P>>64;
13 }
14 inline u32 mul(u32 a,u64 z) {
15     return mul_t(a,trans(z));
16 }

```

7 Poly

7.1 多项式求逆

FILE poly/finv.cpp

LINES 97

USE ntt,多项式求逆

COST $O(n \log n)$

AUTHOR ysy

```

1  #include<iostream>
2  #include<cstdint>

```

```

3 #include<cstring>
4 #define int long long
5 using namespace std;
6 int read(){
7     int x=0,f=1;char ch=getchar();
8     while(ch<'0' || ch>'9'){if(ch=='-')f=-1;ch=getchar();}
9     while(ch>='0' && ch<='9'){x=x*10+ch-'0';ch=getchar();}
10    return x*f;
11 }
12 const int N=4000010,mod=998244353,g=3;
13 int ksm(int p,int k){
14     int x=1;
15     while(k){
16         if(k&1)x=x*p%mod;
17         p=p*p%mod;
18         k>>=1;
19     }
20     return x;
21 }
22 int r[N];
23 void init(int lg){
24     for(int i=0;i<(1<<lg);i++){
25         r[i]=(r[i>>1]>>1)|((i&1)<<(lg-1));
26     }
27 }
28 void ntt(int *a,int n,bool f=0){
29     for(int i=0;i<n;i++){
30         if(i<r[i])swap(a[i],a[r[i]]);
31     }
32     for(int ln=2;ln<=n;ln<=1){
33         int x=ksm(g,(mod-1)/ln),mid=(ln>>1);
34         if(f)x=ksm(x,mod-2);
35         for(int i=0;i<n;i+=ln){
36             int y=1;
37             for(int j=0;j<mid;j++){
38                 int l=a[i+j],r=y*a[i+j+mid]%mod;
39                 a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
40                 y=y*x%mod;
41             }
42         }
43     }
44     if(f){
45         int iv=ksm(n,mod-2);
46         for(int i=0;i<n;i++){
47             a[i]=a[i]*iv%mod;
48         }
49     }
50 }
51 void mul(int *a,int *b,int n){
52     static int c[N],d[N];
53     memset(c,0,sizeof(c));
54     memset(d,0,sizeof(d));
55     int m=(n<<1);
56     for(int i=0;i<n;i++){
57         c[i]=a[i],d[i]=b[i];
58     }
59     ntt(c,m,0);ntt(d,m,0);
60     for(int i=0;i<m;i++){
61         a[i]=c[i]*d[i]%mod;
62     }
63     ntt(a,m,1);
64 }
65 void qinv(int n,int *g){
66     int m=1,lg=0;
67     static int f[N],h[N];
68     memcpy(f,g,sizeof(f));
69     memset(g,0,sizeof(g));
70     g[0]=ksm(f[0],mod-2);
71     while(m<=n){
72         m<<=1,lg++;
73         init(lg+1);
74         memcpy(h,g,sizeof(h));
75         memset(g,0,sizeof(g));
76         for(int i=0;i<m;i++){
77             g[i]=h[i]*2%mod;

```

```

78     }
79     mul(h,h,m);
80     mul(h,f,m);
81     for(int i=0;i<m;i++){
82         g[i]=(g[i]-h[i]+mod)%mod;
83     }
84 }
85 }
86 int n,a[N],b[N];
87 signed main(){
88     n=read()-1;
89     for(int i=0;i<=n;i++){
90         a[i]=read();
91     }
92     qinv(n,a);
93     for(int i=0;i<=n;i++){
94         cout<<a[i]<<" ";
95     }
96     return 0;
97 }

```

7.2 多项式乘法

FILE poly/ntt(ysy).cpp

LINES 73

USE ntt,未经任何优化

COST $O(n\log n)$

AUTHOR ysy

```

1 #include<iostream>
2 #include<cstdio>
3 #define int long long
4 using namespace std;
5 int read(){
6     int x=0,f=1;char ch=getchar();
7     while(ch<'0' || ch>'9'){if(ch=='-')f=-1;ch=getchar();}
8     while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
9     return x*f;
10 }
11 const int N=4000040,mod=998244353,g=3;
12 int n,m,a[N],b[N],r[N];
13 int ksm(int p,int k){
14     int x=1;
15     while(k){
16         if(k&1)x=x*p%mod;
17         p=p*p%mod;
18         k>>=1;
19     }
20     return x;
21 }
22 void init(int lg){
23     for(int i=1;i<(1<<lg);i++){
24         r[i]=((r[i>>1]>>1)|((i&1)<<(lg-1)));
25     }
26 }
27 void ntt(int *a,int lg,int fl=0){
28     int n=(1<<lg);
29     for(int i=0;i<n;i++){
30         if(i<r[i])swap(a[i],a[r[i]]);
31     }
32     for(int ln=2;ln<=n;ln<=1){
33         int mid=(ln>>1),x=ksm(g,(mod-1)/ln);
34         if(fl)x=ksm(x,mod-2);
35         for(int i=0;i<n;i+=ln){
36             int y=1;
37             for(int j=0;j<mid;j++){
38                 int l=a[i+j],r=a[i+j+mid]*y%mod;
39                 a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
40                 y=y*x%mod;
41             }
42         }
43     }
44     if(fl){
45         int iv=ksm(n,mod-2);
46         for(int i=0;i<n;i++){
47             a[i]=a[i]*iv%mod;
48         }
49     }

```

```

50 }
51 void mul(){
52     int lg=__lg(n+m)+1;
53     init(lg);
54     ntt(a,lg);ntt(b,lg);
55     for(int i=0;i<(1<<lg);i++){
56         a[i]=a[i]*b[i]%mod;
57     }
58     ntt(a,lg,1);
59 }
60 signed main(){
61     n=read(),m=read();
62     for(int i=0;i<=n;i++){
63         a[i]=read();
64     }
65     for(int i=0;i<=m;i++){
66         b[i]=read();
67     }
68     mul();
69     for(int i=0;i<=n+m;i++){
70         cout<<a[i]<<" ";
71     }
72     return 0;
73 }

```

7.3 计数常用代码

FILE poly/ntt(yzj).cpp

LINES 74

USE mint 类和 poly 类

COST $O(n \log n)$

AUTHOR yzj

```

1  const int MOD = 998244353;
2
3  // CALC
4  inline void inc (int &x, int y) { x += y - MOD; x += (x >> 31) & MOD; }
5  struct mint {
6      int x;
7      mint (int ix = 0) { x = ix % MOD; if(x < 0) x += MOD; }
8      mint operator += (mint a) { inc (x, a.x); return a; }
9      mint operator -= (mint a) { inc (x, MOD - a.x); return a; }
10     mint operator *= (mint a) { x = 1ll * x * a.x % MOD; return a; }
11     mint friend operator + (mint a, mint b) { a += b; return a; }
12     mint friend operator - (mint a, mint b) { a -= b; return a; }
13     mint friend operator * (mint a, mint b) { a *= b; return a; }
14 } fc[N], ifc[N];
15 inline mint qpow (mint a, int b) { mint res = 1; for(; b; b >>= 1, a *= a) if(b & 1) res *= a; return res; }
16 inline mint qinv (mint a) { return qpow(a.x, MOD - 2); }
17 inline mint binom (int n, int m) { return n < m ? 0 : fc[n] * ifc[m] * ifc[n - m]; }
18 //
19
20 namespace Poly {
21     const int MAXN = 1 << 22;
22     int t, bit, rev[MAXN];
23
24     struct poly : vc <mint> {
25         poly (int n = 0) { resize (n); for (auto &u : (*this)) u = 0; }
26         inline void show () {
27             for (auto u : (*this)) cerr << u.x << " ";
28             cerr << endl;
29         }
30     };
31
32     inline void Resize (poly &a, int n) {
33         while (a.size () < n) a.pb (0);
34         while (a.size () > n) a.pop_back ();
35     }
36     inline void init (int n) {
37         t = 2, bit = 1; while (t < n) t <= 1, ++ bit;
38         rep (i, 0, t - 1) rev[i] = (rev[i] >> 1) >> 1 | ((i & 1) << (bit - 1));
39     }
40     inline void dft (poly &a) {
41         rep (i, 0, t - 1) if (i < rev[i]) swap (a[i], a[rev[i]]);
42         mint o = 1, w = qpow (3, (MOD - 1) >> 1), x, y;
43         for (int m = 1; m < t; m <= 1, w = qpow (3, (MOD - 1) / (m << 1)))
44             for (int i = 0; i < t; i += (m << 1), o = 1) for (int j = 0; j < m; j ++, o = o * w)
45                 x = a[i + j], y = o * a[i + j + m], a[i + j] = x + y, a[i + j + m] = x - y;

```

```

46     }
47     inline void idft (poly &a) {
48         dft (a); auto inv = qinv (t);
49         reverse (a.begin () + 1, a.end ());
50         for (auto &u : a) u *= inv;
51     }
52     poly operator * (poly a, poly b) {
53         int h; init (h = a.size () + b.size () - 1);
54         poly f (t); Resize (a, t); Resize (b, t);
55         dft (a); dft (b);
56         rep (i, 0, t - 1) f[i] = a[i] * b[i];
57         idft (f); Resize (f, h);
58         return f;
59     }
60 }
61
62 int n, c[N], g[N];
63
64 inline Poly :: poly solve (int l, int r) {
65     if (l == r) {
66         Poly :: poly f (c[l] + 1);
67         rep (i, 0, min (c[l], g[l]))
68             if (i & 1) f[c[l] - i] = MOD - binom (c[l], i) * binom (g[l], i) * fc[i];
69             else f[c[l] - i] = binom (c[l], i) * binom (g[l], i) * fc[i];
70         return f;
71     }
72     int mid = ((l + r) >> 1);
73     return solve (l, mid) * solve (mid + 1, r);
74 }

```

7.4 DIF-DIT NTT & All op

FILE poly/poly.cpp

LINES 189

USE 效率较高的 NTT

COST 除『』外, 均为 $O(n \log n)$

AUTHOR ppip

```

1 using LL=long long;
2 #define ensure(x) do { \
3     if (!(x)) { \
4         fprintf(stderr, "ensure failed: %s, line %d\n", #x, __LINE__); \
5         exit(1); \
6     } \
7 } while(0)
8 struct poly {
9     static const int mod=998244353;
10    vector<LL>a;
11    static vector<LL>w;
12
13    poly(){}
14    poly(int n):a(n){}
15    poly(vector<LL> v):a(move(v)){}
16    poly(initializer_list<LL> v):a(v){}
17
18    int size()const{return a.size();}
19    LL& operator[](int i){return a[i];}
20    const LL& operator[](int i)const{return a[i];}
21    void resize(int n){a.resize(n);}
22
23    static LL qp(LL a,LL n=mod-2){
24        LL r=1;
25        for(;n>>=1;a=a*a%mod)if(n&1)r=r*a%mod;
26        return r;
27    }
28
29    static int up(int n){
30        return n<=1?1:1<<(__lg(n-1)+1);
31    }
32
33    static void init(int n){
34        ensure(n>0&&!(n&(n-1))&&n<=(1<<23));
35        int m=n>>1,k=w.size();
36        if(k>=m)return;
37        w.resize(m);
38        for(;k<m;k<=1){
39            LL r=qp(31,1<<(__lg(k)));
40            for(int i=0;i<k;i++)w[k+i]=w[i]*r%mod;

```

```

41     }
42 }
43
44 static void dif(vector<LL>&a){
45     int n=a.size();
46     init(n);
47     for(int len=n>>1;len;len>>=1){
48         int o=0;
49         for(int l=0;l<n;l+=len<<1,++o){
50             LL r=w[o];
51             for(int i=l;i<l+len;i++){
52                 LL x=a[i],y=r*a[i+len]%mod;
53                 a[i]=x+y;
54                 if(a[i]>=mod)a[i]-=mod;
55                 a[i+len]=x-y;
56                 if(a[i+len]<0)a[i+len]+=mod;
57             }
58         }
59     }
60 }
61
62 static void dit(vector<LL>&a){
63     int n=a.size();
64     init(n);
65     for(int len=1;len<n;len<=<1){
66         int o=0;
67         for(int l=0;l<n;l+=len<<1,++o){
68             LL r=w[o];
69             for(int i=l;i<l+len;i++){
70                 LL x=a[i],y=a[i+len];
71                 a[i]=x+y;
72                 if(a[i]>=mod)a[i]-=mod;
73                 a[i+len]=(x-y+mod)*r%mod;
74             }
75         }
76     }
77 }
78
79 vector<LL> val(int n=0)const{
80     if(!n)n=up(size());
81     vector<LL>f(n);
82     copy_n(a.begin(),min(n,size()),f.begin());
83     dif(f);
84     return f;
85 }
86
87 static poly from(vector<LL>f){
88     if(f.empty())return {};
89     int n=f.size();
90     dit(f);
91     reverse(f.begin()+1,f.end());
92     LL iv=mod-(mod-1)/n;
93     for(LL&x:f)x=x*iv%mod;
94     return poly(move(f));
95 }
96
97 friend poly operator*(const poly&A,const poly&B){
98     if(!A.size()||!B.size())return {};
99     int s=A.size()+B.size()-1;
100     if(min(A.size(),B.size())<=32){
101         poly c(s);
102         for(int i=0;i<A.size();i++)for(int j=0;j<B.size();j++)
103             c[i+j]=(c[i+j]+A[i]*B[j])%mod;
104         return c;
105     }
106     int n=up(s);
107     auto a=A.val(n);
108     if(&A==&B){
109         for(LL&x:a)x=x*x%mod;
110     }else{
111         auto b=B.val(n);
112         for(int i=0;i<n;i++)a[i]=a[i]*b[i]%mod;
113     }
114     poly c=from(move(a));
115     c.resize(s);

```

```

116     return c;
117 }
118
119 friend poly operator+(poly A,const poly&B){
120     if(A.size()<B.size()) A.resize(B.size());
121     for(int i=0;i<B.size();++i){
122         A[i]+=B[i];
123         if(A[i]>=mod) A[i]-=mod;
124     }
125     return A;
126 }
127 friend poly operator-(poly A,const poly&B){
128     if(A.size()<B.size()) A.resize(B.size());
129     for(int i=0;i<B.size();++i){
130         A[i]-=B[i];
131         if(A[i]<0) A[i]+=mod;
132     }
133     return A;
134 }
135
136 poly& operator+=(const poly&b){
137     return *this=*this+b;
138 }
139 poly& operator-=(const poly&b){
140     return *this=*this-b;
141 }
142 poly& operator*=(const poly&b){
143     return *this=*this*b;
144 }
145
146 friend poly inv(const poly &f,int n=-1) {
147     ensure(f.size() && f[0]);
148     if (n==-1) n=(f.size()-1);
149     poly F{f[0]},G{qp(F[0])};
150     for (int i{2};(i>>1)<=n;i<=1) {
151         // FG=1+E
152         // G1=G(1-E)=G(1-(FG-1))=G(2-FG)
153         F.resize(i);
154         for (int j{i/2};j<i&&j<=n;++j) F[j]=f[j];
155         G=G*(poly{2}-F*G);
156         G.resize(i);
157     }
158     G.resize(n+1);
159     return G;
160 }
161 friend poly dev(const poly &f) {
162     if (f.size()<=1) return poly{0};
163     poly g(f.size()-1);
164     for (int i=1;i<f.size();++i)
165         g[i-1]=f[i]*i%mod;
166     return g;
167 }
168 friend poly inte(const poly &f) {
169     poly g(f.size()+1);
170     g[0]=0;
171     vector<LL> iv(f.size()+1);
172     if (f.size()) iv[1]=1;
173     for (int i=2;i<=f.size();++i)
174         iv[i]=mod-mod/i*iv[mod%i]%mod;
175     for (int i=0;i<f.size();++i)
176         g[i+1]=f[i]*iv[i+1]%mod;
177     return g;
178 }
179 friend poly ln(const poly &f,int n=-1) {
180     ensure(f[0]==1);
181     // G'=(LnF)'=F'/F
182     // G=int(F'*F^-1)
183     if (n==-1) n=f.size();
184     auto g{inte(dev(f)*inv(f))};
185     g.resize(n);
186     return g;
187 }
188 };
189 vector<LL> poly::w{1};

```

8 String

8.1 广义 SAM

FILE string/exsam.cpp

LINES 62

USE 建立多串 SAM

COST $O(n \cdot C)$

AUTHOR ysy

```

1 #include<iostream>
2 #include<cstdio>
3 #include<cstring>
4 #define ll long long
5 using namespace std;
6 int read(){
7     int x;
8     scanf("%d",&x);
9     return x;
10 }
11 const int N=2000020;
12 struct SAM{
13     int fa[N],ch[N][26],ln[N],ct=1,la=1;
14     void ins(int c){
15         int p=la;
16         if(ch[p][c]){
17             int q=ch[p][c];
18             if(ln[q]==ln[p]+1)la=q;
19             else{
20                 int nq=++ct;ln[nq]=ln[p]+1,la=nq;
21                 memcpy(ch[nq],ch[q],sizeof(ch[q]));
22                 fa[nq]=fa[q];fa[q]=nq;
23                 for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=nq;
24             }
25             return;
26         }
27         int np=++ct;ln[np]=ln[p]+1,la=np;
28         for(;p&&!ch[p][c];p=fa[p])ch[p][c]=np;
29         if(!p)fa[np]=1;
30         else{
31             int q=ch[p][c];
32             if(ln[q]==ln[p]+1)fa[np]=q;
33             else{
34                 int nq=++ct;ln[nq]=ln[p]+1;
35                 memcpy(ch[nq],ch[q],sizeof(ch[q]));
36                 fa[nq]=fa[q];fa[q]=fa[np]=nq;
37                 for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=nq;
38             }
39         }
40     }
41     ll calc(){
42         ll ans=0;
43         for(int i=2;i<=ct;i++){
44             ans+=ln[i]-ln[fa[i]];
45         }
46         return ans;
47     }
48 }sa;
49 int n,m;
50 char c[N];
51 int main(){
52     n=read();
53     for(int i=1;i<=n;i++){
54         scanf("%s",c+1);
55         m=strlen(c+1),sa.la=1;
56         for(int j=1;j<=m;j++){
57             sa.ins(c[j]-'a');
58         }
59     }
60     cout<<sa.calc()<<"\n"<<sa.ct;
61     return 0;
62 }

```

8.2 manacher

FILE string/manacher.cpp

LINES 13

USE 线性求回文子串数量

COST $O(n)$

AUTHOR yzj

```

1 namespace Manacher {
2     int r[N], L, R, res;
3     inline int manacher(char *s, int n) {
4         res = L = R = 0;
5         rep(i, 1, n) {
6             r[i] = i <= R ? min(r[R + L - i], R - i) : 0;
7             while(i + r[i] + 1 <= n && i - r[i] - 1 > 0 && s[i + r[i] + 1] == s[i - r[i] - 1]) ++r[i];
8             if(i + r[i] > R) L = i - r[i], R = i + r[i];
9             res = max(res, r[i] / 2 * 2 + (i % 2 == 0));
10        }
11        return res;
12    }
13 }

```

8.3 PAM

FILE string/pam.cpp

LINES 27

USE 回文树

COST $O(n)$

AUTHOR yzj

```

1 namespace PAM {
2     int ch[N][26], fail[N], len[N], cnt[N], last, tot;
3     inline int getfail(int p, int x) {
4         while(str[x - len[p] - 1] != str[x]) p = fail[p];
5         return p;
6     }
7     inline void insert(int x) {
8         int c = str[x] - 'a', p = getfail(last, x);
9         if(!ch[p][c]) {
10            len[++tot] = len[p] + 2;
11            if(len[tot] == 1) fail[tot] = 0;
12            else {
13                int q = getfail(fail[p], x);
14                fail[tot] = ch[q][c];
15            }
16            ch[p][c] = tot;
17        }
18        last = ch[p][c]; cnt[last]++;
19    }
20    inline void build() {
21        len[0] = 0; fail[0] = 1;
22        len[1] = -1; fail[1] = 1;
23        last = 0; tot = 1;
24        for(int i = 1; i <= n; i++) insert(i);
25        for(int i = tot; i > 1; i--) cnt[fail[i]] += cnt[i];
26    }
27 }

```

8.4 SA

FILE string/sa(oscar).cpp

LINES 41

USE 后缀排序并求出 height 数组

COST $O(n \log n)$

AUTHOR yzj

```

1 namespace SA {
2     int sa[N], rk[N], tmp[N], id[N], buc[N], height[N], idx;
3     inline void suffix_sort() {
4         for(int i = 0; i < 256; i++) buc[i] = 0;
5         for(int i = 1; i <= n; i++) buc[(unsigned char)str[i]]++;
6         for(int i = 1; i < 256; i++) buc[i] += buc[i - 1];
7         for(int i = n; i; i--) sa[buc[(unsigned char)str[i]]--] = i;
8         rk[sa[1]] = idx = 1;
9         for(int i = 2; i <= n; i++)
10            rk[sa[i]] = str[sa[i]] == str[sa[i - 1]] ? idx : ++idx;
11
12        for(int w = 1; idx < n; w <= 1) {
13            int p = 0;
14            for(int i = n - w + 1; i <= n; i++) id[++p] = i;
15            for(int i = 1; i <= n; i++) if(sa[i] > w) id[++p] = sa[i] - w;
16
17            for(int i = 1; i <= idx; i++) buc[i] = 0;

```

```

18     for(int i = 1; i <= n; i++) buc[rk[id[i]]]++;
19     for(int i = 1; i <= idx; i++) buc[i] += buc[i - 1];
20     for(int i = n; i; i--) sa[buc[rk[id[i]]]--] = id[i];
21
22     for(int i = 1; i <= n; i++) tmp[i] = rk[i];
23     rk[sa[1]] = idx = 1;
24     for(int i = 2; i <= n; i++) {
25         int x = sa[i], y = sa[i - 1];
26         rk[x] = tmp[x] == tmp[y] &&
27             (x + w > n ? 0 : tmp[x + w]) == (y + w > n ? 0 : tmp[y + w])
28             ? idx : ++idx;
29     }
30 }
31
32 for(int i = 1, k = 0; i <= n; i++) {
33     if(rk[i] == 1) { k = 0; continue; }
34     if(k) k--;
35     int j = sa[rk[i] - 1];
36     while(i + k <= n && j + k <= n && str[i + k] == str[j + k]) k++;
37     height[rk[i]] = k;
38 }
39 height[1] = 0;
40 }
41 }

```

8.5 后缀数组

FILE string/sa.cpp

LINES 34

USE 倍增算法求 sa, rk, height, 字符必须不超过 '~'

COST $O(n \log n)$

AUTHOR ppip

```

1 char s[N+5];
2 int id[N*2+5], sa[N+5], buc[N+5], h[N+5], rk[N+5];
3 int main() {
4     // read s
5     int n(strlen(s+1));
6     for (int i{1}; i<=n; ++i) ++buc[s[i]];
7     partial_sum(buc+1, buc+1+'~', buc+1);
8     for (int i{n}; i>=1; --i) sa[buc[s[i]]--]=i;
9     int m{0};
10    for (int i{1}; i<=n; ++i) rk[sa[i]] = m += (s[sa[i]] != s[sa[i-1]]);
11    fill_n(buc+1, '~', 0);
12    for (int w{1}; m<n; w<=1) {
13        int cnt{0};
14        for (int i{0}; i<w; ++i) id[++cnt] = n-i;
15        for (int i{1}; i<=n; ++i) {
16            if (sa[i]>w) id[++cnt] = sa[i]-w;
17            ++buc[rk[i]];
18        }
19        partial_sum(buc+1, buc+1+m, buc+1);
20        for (int i{n}; i>=1; --i) sa[buc[rk[id[i]]]--]=id[i];
21        fill_n(buc+1, m, 0);
22        copy_n(rk+1, n, id+1);
23        m=0;
24        for (int i{1}; i<=n; ++i)
25            rk[sa[i]] = m += (id[sa[i]] != id[sa[i-1]] || id[sa[i]+w] != id[sa[i-1]+w]);
26    }
27    // for (int i{1}; i<=n; ++i) cout<<sa[i]<<" ";
28    for (int i{1}, k{0}; i<=n; ++i) {
29        k-=!!k;
30        while (s[i+k]==s[sa[rk[i]-1]+k]) ++k;
31        h[rk[i]]=k;
32    }
33    return 0;
34 }

```

8.6 SAM

FILE string/sam.cpp

LINES 36

USE Magic automaton

COST $O(n)$

AUTHOR ppip

```

1 constexpr int N(1e6), S{26};
2 struct SAM {
3     int cnt, cur, n;
4     struct node {
5         int len, link;

```

```

6      array<int,S> ch;
7  } t[N*2+5];
8  SAM() {cur=cnt=1;fill(t[0].ch.begin(),t[0].ch.end(),1);t[0].len=-1;}
9  void extend(int c) {
10     ++n;
11     int x{cur},p{++cnt};
12     t[p].len=n;
13     while (!t[x].ch[c]) t[x].ch[c]=p,x=t[x].link;
14     int q{t[x].ch[c]};
15     t[p].link=q;
16     if (t[x].len+1!=t[q].len) {
17         int nq{++cnt};
18         t[nq].ch=t[q].ch;
19         t[nq].link=t[q].link;
20         t[nq].len=t[x].len+1;
21
22         t[p].link=t[q].link=nq;
23         while (t[x].ch[c]==q) t[x].ch[c]=nq,x=t[x].link;
24     }
25     cur=p;
26 }
27 void print() {
28     return;
29     for (int i{1};i<=cnt;++i)
30         for (int j{0};j<S;++j)
31             if (t[i].ch[j])
32                 cerr<<i<<" "<<t[i].ch[j]<<" "<<char(j+'a')<<endl;
33     for (int i{cur};i=t[i].link) cerr<<i<<" ";
34     cerr<<endl<<endl;
35 }
36 } T;

```

8.7 后缀树

FILE string/sft.cpp

LINES 39

USE Ukkonen 在线构造隐式后缀树

COST build O(n)

AUTHOR ppip

```

1  constexpr int N(1e6),inf(1e7),RNG{26+26+10};
2  struct Suft {
3      int ch[N*2+5][RNG];
4      int st[N*2+5],len[N*2+5],link[N*2+5];
5      char s[N+5];
6      int now{1},rem{0},n{0},tot{1};
7      Suft() {len[0]=inf;}
8      int new_node(int l,int le) {
9          ++tot;st[tot]=l;len[tot]=le;
10         return tot;
11     }
12     void extend(int x) {
13         s[++n]=x;rem++;
14         for (int lst{1};rem;) {
15             while (rem>len[ch[now][s[n-rem+1]])
16                 rem=len[now=ch[now][s[n-rem+1]]];
17             int &v{ch[now][s[n-rem+1]],c{s[st[v]+rem-1]}};
18             if (!v||x==c) {
19                 link[lst]=now;lst=now;
20                 if (!v) v=new_node(n,inf);
21                 else break;
22             } else {
23                 int u{new_node(st[v],rem-1)};
24                 ch[u][c]=v;ch[u][x]=new_node(n,inf);
25                 st[v]+=rem-1;len[v]-=rem-1;
26                 link[lst]=v=u;lst=u;
27             }
28             if (now==1) --rem;
29             else now=link[now];
30         }
31     }
32     void search(int u,int dep=0) {
33         if (st[u]+len[u]-1==n&&st[u]-dep!=n) cout<<st[u]-dep<<" ";
34         else dep+=len[u];
35         for (int i{0};i<RNG;++i)
36             if (ch[u][i]) search(ch[u][i],dep);
37     }

```

```
38  
39 } T;
```

警钟长鸣

读题与建模

！ 多模拟样例

实现与边界

！ 任意范围修改、查询时，考虑复杂度会不会退化

思考误区

！ 发现新性质后，回头检查旧做法能否简化或是否已经不成立

！ DP 先想清楚有哪些可靠的状态和存储方式，再设计转移

提交之前

！ 检查多测清空

！ 检查 long long 和取模

！ TLE 时重新检查实际复杂度，尤其注意分治树等结构是否退化